

LVDS 板卡以太网 UDP 控制协议说明文档

一、基本信息

- 协议类型：UDP 单播
- ip:默认 192.168.1.10
- 默认端口：5555
- 最大数据长度：1500 字节以内（包含协议头）
- 字节序：小端格式（little endian）
- 校验方式：数据尾部 1 字节校验（checksum()）

二、通用数据帧格式

PC 向服务器发出

| 消息长 | 预留段 | 消息代码 | 消息体 | 校验和 |

每条命令格式如下：

字段	长度	说明
消息长	2 字节	消息总长度（含校验）
预留段	2 字节	填 0x0000
消息码	1 字节	功能码（msg_cmd）
消息体	N 字节	命令参数或数据
校验和	1 字节	所有字节异或校验

校验和函数如下

```
uint8_t checksum(uint8_t * ptr, int16_t cnt)
{
    int16_t i = 0;
    uint8_t calc=0;
    for(i=0;i<cnt;i++)
    {
        calc=calc + ptr[i];
    }

    return calc=~calc&0xff;
}
```

三、命令字说明 (msg_cmd)

0x10: 读取单个内存地址

说明: 读取指定地址的 4 字节内容

总长度: 10

示例 (读取地址 0x80070000):

PC 发送: 10 字节

| 消息长 | 预留段 | 消息代码 | 消息体(地址) | 校验和 |

| 0A 00 | 00 00 | 10 | 00 00 07 80 | 5E

板卡返回: 14 字节

| 消息长 | 预留段 | 消息代码 | 消息体(地址) + 实际读值 | 校验和 |

0E 00 00 00 10 00 00 07 80 16 07 07 25 11

0x11: 写入单个内存地址

说明: 向板卡指定地址写入 4 字节数据

总长度: 14

示例 (写入地址 0x80070000):

PC 发送: 14 字节

| 消息长 | 预留段 | 消息代码 | 消息体(地址) + 写入值 | 校验和 |

0E 00 00 00 11 00 00 07 80 16 07 07 25 10

板卡返回: 14 字节

| 消息长 | 预留段 | 消息代码 | 消息体(地址) + 从写入地址读回的值 | 校验和 |

0E 00 00 00 11 00 00 07 80 16 07 07 25 10

0x12: 读取连续内存块

说明: 从指定地址连续读取多个 4 字节数据 (总共 N 个)

总长度: 10 字节 (头) + 返回 N×4 字节数据

示例 (读取地址 0x80070000 起, 3 个 32bit):

PC 发送: 14 字节

| 消息长 | 预留段 | 消息代码 | 起始地址 + 数量 | 校验和 |

0E 00 00 00 12 00 00 07 80 03 00 00 00 9C

板卡返回: 14 + 12 = 26 字节

| 消息长 | 预留段 | 消息代码 | 起始地址 + 读回值 | 校验和 |

1A 00 00 00 12 00 00 07 80 XX XX XX XX YY YY YY YY ZZ ZZ ZZ ZZ CRC

0x13: 写入连续内存块

说明: 从指定地址连续写入多个 4 字节数据

总长度: 14 + N×4 字节

示例 (向 0x80070000 写入两个值: 0x11111111, 0x22222222):

PC 发送: 22 字节

| 消息长 | 预留段 | 消息代码 | 地址 + 数量 + 数据 | 校验和 |
16 00 00 00 13 00 00 07 80 02 00 00 00 11 11 11 11 22 22 22 22 CRC

板卡返回: 14 字节

| 消息长 | 预留段 | 消息代码 | 地址 + 返回写入数量 | 校验和 |
0E 00 00 00 13 00 00 07 80 02 00 00 00 CRC

0x20: 读取 I2C 寄存器

总长度: 12 字节

作用: 用于读取板卡 I2C 总线上指定设备寄存器的数值。支持读取 1~255 字节数据, 支持 8 位或 16 位寄存器地址。

PC 发送格式 (共 12 字节)

[0-1]	0x0C	总长度 (12 字节)
[2-3]	0x00 0x00	保留 (可用于通道或标识)
[4]	0x20	命令码: I2C 读取 (0x20)
[5]	通道号	表示哪个 I2C 控制器 (例如: 0、1)
[6]	从机地址	I2C 从设备地址 (7 位地址, 低位有效)
[7]	读取字节数	要读取的寄存器地址
[8-9]	寄存器地址	一次读取的数据长度
[10]	地址宽度标志	0 表示 16 位寄存器地址, 1 为 8 位地址
[11]	CRC 校验	校验字节

示例: 读取 I2C 通道 1、从机地址 0x48、寄存器 0x0000, 读取 1 字节

PC 发送: 12 字节

0C 00 00 00 20 01 48 01 00 00 00 89

板卡返回: 14 字节

0E 00 00 00 20 01 48 01 00 00 00 90 90 F7

[IIC Read] ch: 1, dev_addr: 0x48, reg: 0x0000, len: 1 → data: [0x90]

字节数 11 代表读取成功与否, 成功为 0, 失败为 1

字节数 12 代表读取的寄存器值

字节数 12 代表校验字节

0E 00 00 00 20 01 40 01 00 00 00 01 00 8E

[IIC Read] ch: 1, dev_addr: 0x40, reg: 0x0000, len: 1 → read failed!

当前通道 I2C 总线上不存在相应从机设备会返回失败响应

读取 10 个数据

PC 发送: 0C 00 00 00 20 01 48 0A 00 00 00 80

板卡返回：17 00 00 00 20 01 48 0A 00 00 00 00 90 02 F3 50 00 C0 DF A7 97 35 8E
[IIC Read] ch: 1, dev_addr: 0x48, reg: 0x0000, len: 10 → data: [0x90, 0x02, 0xf3, 0x50, 0x00, 0xc0, 0xdf, 0xa7, 0x97, 0x35]

0x21：I2C 写入命令

总长度：12 字节
作用：向板卡 I2C 从设备的指定寄存器地址写入数据

写入仅为单个字节数据写入，暂不支持连续写

PC 发送格式（共 12 字节）

[0-1]	0x0C	总长度（12 字节）
[2-3]	0x00 0x00	保留（可用于通道或标识）
[4]	0x21	命令码：I2C 读取（0x20）
[5]	通道号	表示哪个 I2C 控制器（例如：0、1）
[6]	从机地址	I2C 从设备地址（7 位地址，低位有效）
[7-8]	寄存器地址	要写入的寄存器地址
[9]	地址宽度标志	0 表示 16 位寄存器地址，1 为 8 位地址
[10]	状态码	板卡发送 0x00 表示成功，0x01 表示失败
[11]	写入的数据	写入寄存器的值
[12]	CRC 校验	校验字节

示例：写入通道 1、从机地址 0x48、寄存器 0x0000，写入 1 字节数据 0x12

PC 发送（13 字节）：

0D 00 00 00 21 01 48 00 00 00 00 90 F8

板卡返回（成功，固定 13 字节）：

0D 00 00 00 21 01 48 00 00 00 00 90 F8

说明：状态字节为 0 表示写入成功

板卡返回（失败）：

0D 00 00 00 21 01 40 00 00 00 01 90 FF

说明：状态字节为 1 表示写入失败（如从机地址不存在）

0x30：图像比较命令（待完善）

作用：向板卡图像比较 IP 设置参数控制

cmd_index	示例数据	说明
0	08 00 00 00 30 00 01 C6	关闭控制开关
1	08 00 00 00 30 01 01 C5	打开控制开关
2	09 00 00 00 30 02 01 05 BE	设置阈值为 5
3	08 00 00 00 30 03 01 C3	获取分辨率与帧率
4	08 00 00 00 30 04 01 C2	获取状态

标红处字节位对应板卡内部图像通道

其中 cmd_index 为 0、1、2 时板卡返回与发送字节一致

cmd_index 为 4 示例如下：

PC 发送 8 字节：08 00 00 00 30 03 01 C3

板卡响应 20 字节：

14 00 00 00 30 04 00 00 0F 00 00 70 08 00 00 24 00 00 00 0C

其中[7-10] 为视频列，[11-15] 为视频行，[16-19] 为帧率

板卡返回图像通道 1 的视频信息：3840*2160 36fps

cmd_index 为 5 示例如下：

PC 发送 8 字节：08 00 00 00 30 04 01 C2

板卡响应 48 字节：

其中[7-10]为视频列，[11-15]为视频行，[16-19]为帧率，[20-23]为状态码，[24-27]帧总数，
[28-31]为设置比对 RGB 值，[32-35]为当前帧输入像素点，[36-39]为错误帧像素点，[40-43]
为输入帧像素点，[44-47]为像素阈值

30 00 00 00 30 05 00 00 0F 00 00 70 08 00 00 24 00 00 00 02 00 00 00 78 8A 01 00 00 00 FF
00 00 00 FF 00 00 00 FF 00 00 00 00 00 05 00 00 00 E8

分辨率：3840 x 2160

帧率：36 fps

状态码：2

帧总数：100984

对比像素点：FF0000

当前帧输入像素点：FF0000

错误帧像素点：FF0000

输入帧像素点：0

像素阈值：5

状态码 bit0 为控制开关，值 1 打开，值 0 关闭

状态码 bit1 为状态标志，值 1 表示像素点比对失败，值 0 表示成功

帧总数为板卡内部视频接收总帧数

对比像素点为设置比对像素点值

错误帧像素点与输入帧像素点分别表示像素点比对失败时记录的对比值与输入值

只要像素点对比值与像素输入值差值超过阈值则记录像素点比对失败

2026/1/22 修改

将第 7 字节改为通道，通道从 1 开始计

0x40： 变量配置与读取

cmd_index	示例数据	说明
0	08 00 00 00 40 00 00 BF	获取板卡硬件版本
1	08 00 00 00 40 01 00 BE	获取板卡软件版本
2	08 00 00 00 40 02 00 BD	获取 flash 地址
3	09 00 00 00 40 03 00 BC	获取 flash 类型
4	08 00 00 00 40 04 00 BB	获取 flash 大小
...	保留	
10	08 00 00 00 40 0A 00 B5	获取板卡网络当前设置
11		设置板卡网络保存 EEPROM
12	08 00 00 00 40 0C 00 B3	获取板卡网络默认设置

板卡响应：

返回的字节数据中[0-6]为固定帧格式，[7-n]为数据信息

cmd_index 为 0、1、2、3、4 时，返回类型如下

Received board_cmd_data: "0C 00 00 00 40 00 00 16 07 07 25 6A "
Received board_cmd_data: "0C 00 00 00 40 01 00 21 23 07 25 42 "
Received board_cmd_data: "0C 00 00 00 40 02 00 00 00 30 00 81 "
Received board_cmd_data: "0E 00 00 00 40 03 00 4D 49 43 52 4F 4E E6 "
Received board_cmd_data: "0C 00 00 00 40 04 00 00 80 00 00 2F "

cmd_index 为 10、12 时，返回 36 字节

其中[7-10]为 4 字节 IP 地址，[11-15]为 4 字节子网掩码，[16-19]网关地址，[20-23]为板卡 mac 地址

示例如下：

Received board_cmd_data: "24 00 00 00 40 0A 00 29 86 6B 11 BC FD 9F
00 C0 A8 01 0A FF FF FF 00 C0 A8 01 01 10 0A 35 00 01 02 00 00 E2 "
IP: "192.168.1.10"
Mask: "255.255.255.0"
Gateway: "192.168.1.1"
MAC: "10:0A:35:00:01:02"

Received board_cmd_data: "24 00 00 00 40 0C 00 00 00 00 00 00 00 00
00 C0 A8 01 0A FF FF FF 00 C0 A8 01 01 10 0A 35 00 01 02 FF FF 65 "
IP: "192.168.1.10"
Mask: "255.255.255.0"
Gateway: "192.168.1.1"
MAC: "10:0A:35:00:01:02"

Sent 36 bytes to "192.168.1.10" : 5555

发送设置板卡网络设置命令如下：

Sent 36 bytes to "192.168.1.10" : 5555

Send cmd 11 len: 36 data: "24 00 00 00 40 0B 00 00 00 00 00 00 00 00 00 C0 A8 01 0A FF FF FF 00 C0 A8 01 01 10 0A 35 00 01 02 00 00 64"

receive len 9

Received board_cmd_data: "09 00 00 00 40 0B 00 00 AB "

0x80：图像/视频控制命令

作用：控制图像传输开启与关闭，传输的通道，传输视频还是图片。

板卡不发送响应

PC 发送格式（共 8 字节）

[0-1]	0x0C	总长度（8 字节）
[2-3]	0x00 0x00	保留（可用于通道或标识）
[4]	0x80	命令码：I2C 读取 (0x20)
[5]	参数 1（通道）	表示图像通道
[6]	参数 2（操作类型）	控制行为（例如开关图像、视频）
[7]	CRC 校验	校验字节

各子命令说明（对应[5]参数 1）

值	含义
0x00	关闭所有通道图片发送
0xFF	所有通道开始图片发送
0x01-N	指定通道（编号从 1 开始）发送

各子命令说明（对应[6]参数 2）

0x00: 关闭所有图像显示

说明:

关闭所有图像通道的传输（图片和视频）。

PC 发送 8 字节: 08 00 00 00 80 00 00 77

0x01: 发送图片

控制通道开始发送一帧静态图像。

PC 发送 8 字节: 08 00 00 00 80 FF 01 77

板卡所有图像通道均发送图片到上位机

0x02: 发送视频

PC 发送 8 字节: 08 00 00 00 80 FF 02 76

板卡所有图像通道均连续发送视频到上位机

0x03: 发送错误帧图片

待完善

0x04: 切换视频源（仅在板卡内部多通道图像源时生效）

PC 发送 8 字节: 08 00 00 00 80 01 04 ? ?

切换第一通道

PC 发送 8 字节: 08 00 00 00 80 02 04 ? ?

切换第二通道

PC 发送 8 字节: 08 00 00 00 80 03 04 ? ?

切换第二通道

图像包发送格式

//一个完整的 udp 图像报文由两部分组成:

```
/**
 *-----
 *| packet_header(消息头) (sizeof(packet_header)字节) | 有效图片数据
(1200 字节)|
 *-----
 * */
```

包头定义如下

```

// packet_header.h
typedef struct packet_header
{
    u32 HEAD_ID; // 0xAA5555AA
    u32 Channel_ID; // 图像通道
    u32 SEND_FLAG; // 发送标志, 0: 图片, 1: 视频
    u32 Width; // 图像宽度
    u32 Height; // 图像高度
    u32 total; // 一张图片总字节数 例如 1920*1080*3
    u32 offset; // 当前数据偏移量 例如, 第一帧数据为0, 第n帧数据为 (n-1)*framesize
    u32 picseq; // 图片序号, 第几张图片
    u32 frameseq; // 一张图片发送的帧序号, 当前图片的第n帧, 从1~ 1920*3
    u32 framesize; // 当前帧图片数据大小 例如每一次都发有效图片数据大小为1200
} packet_header;

```

包头共包含 40 字节数据，图像数据 1200 字节，每一帧图像包共 1240 字节数据
若传输的一帧图片为 1920*1080，则共需要传送 $1920 \times 1080 \times 3 / 1200 = 5184$ 帧

上位机接收处理流程

上位机 udp 接收需要分析数据是功能控制码还是图像包，根据帧头进行判断
图像数据包需要进行：

- 图像数据重组（多帧拼接成完整图像）
- 异常检测与容错处理
- OpenCV 格式转换（BGR → RGB）
- 写入共享内存与保存文件
- 图像显示

图像数据重组逻辑：

1. 提取数据头部信息 `packet\_header`：
 - * 通道 ID
 - * 当前帧编号 frameseq
 - * 总大小、图片分包数量等
2. 判定是否为新一帧（picseq 变更）
 - * 是 → 清空状态并记录新序号
3. 判断是否为首帧分段（frameseq == 1）
 - * 是 → 解析图片尺寸并初始化缓存
4. 判断是否为末段：
 - * 如果当前包为最后一个（ $\text{total} / \text{framesize}$ ）

- * 拷贝数据
- * 缓存大小与总大小一致

5. 正常中间段处理:

- *接收自增 frameseq 进行顺序拼接
- * 若 frameseq 异常乱序, 丢弃该帧并复位状态重新接收